

Run your neural nets anywhere™

Or, an ONNX and ort exploration

Kei Tang

Recurse Center

2026-10-14

- So….
- You have trained a cool deep learning model using Pytorch (or your favorite framework). Now you want to run it on as many devices as possible: Laptop, phone, Raspberry Pi…
- But…
- Installing Pytorch on all these different devices is (´;﹏;`)[1]
- Is there a simple solution?

[1] There is `executorch` but it involves writing platform-specific code

1.2 Introducing, ONNX

ONNX (Open Neural Network Exchange) A common format to represent neural networks, designed to facilitate exchange of models across different frameworks.

Workflow:

- **Convert** model from the training framework to ONNX.
- Run the model on devices with ONNX **runtime**.
- Use different **execution providers (EPs)** for different hardware for acceleration.

1.3 How ONNX works

- A neural network is a computational graph. Data flows through nodes, which represent **Operations**.
- The "list" of supported operators is long: Convolution, Attention, Pooling…
- As long as you can execute the graph of operations, you can run model inference!

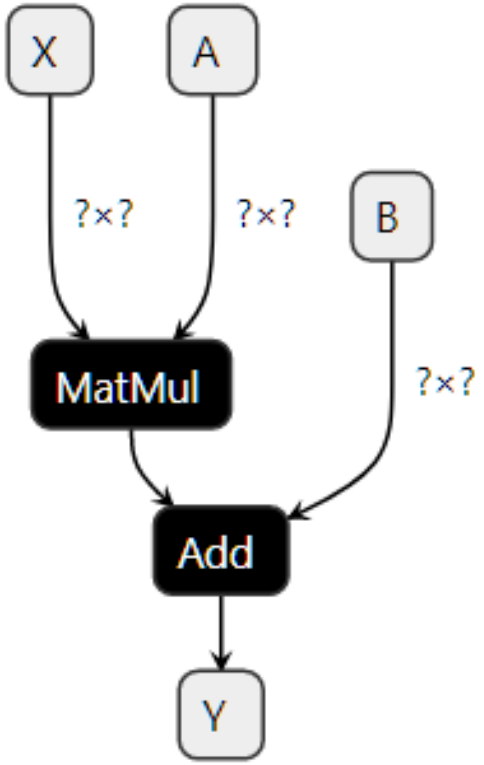


Figure 1: Linear regression as ONNX graph

1.4 Exporting & Visualizing

- Exporting a Pytorch model:

```
torch_model = ImageClassifierModel()
example_inputs = (torch.randn(1, 1, 32, 32),)
onnx_program = torch.onnx.export(torch_model, example_inputs,
dynamo=True)
onnx_program.save("image_classifier_model.onnx")
```

- Visualizing: https://netron.app/

1.5 Run your neural nets anywhere (really)

CPU	GPU	IoT/Edge/Mobile	Other
Default CPU	NVIDIA CUDA	Intel OpenVINO	Rockchip NPU (*preview*)
Intel DNNL	NVIDIA TensorRT	Arm Compute Library (*preview*)	Xilinx Vitis-AI (*preview*)
TVM (*preview*)	DirectML	Android Neural Networks API	Huawei CANN (*preview*)
Intel OpenVINO	AMD MIGraphX	Arm NN (*preview*)	AZURE (*preview*)
XNNPACK	Intel OpenVINO	CoreML (*preview*)	
AMD ROCm(*deprecated*)	Qualcomm QNN	XNNPACK	
	WebGPU		

Figure 2: Execution Providers

2.1 Rust runtime for ONNX

ort Rust gateway to ONNX Runtime, supporting using both default ONNX runtime (in C++) and other Rust-native backends.



Figure 3: Cute icon!

3.1 I want to see a demo!

- My laptop is not suitable for this[1]!

[1] No dGPU

[2] Backends on Strix Halo is a bit unstable

[3] CPU Acceleration, requires compiling ONNX runtime from scratch

[4] <100ms is acceptable for use in the app, so it's kinda useless

3.1 I want to see a demo!

- My laptop is not suitable for this[1]!
- My desktop is also not suitable for this[2]!

[1] No dGPU

[2] Backends on Strix Halo is a bit unstable

[3] CPU Acceleration, requires compiling ONNX runtime from scratch

[4] <100ms is acceptable for use in the app, so it's kinda useless

3.1 I want to see a demo!

- My laptop is not suitable for this[1]!
- My desktop is also not suitable for this[2]!
- So I got XNNPACK EP working on my desktop[3].
- The LSTM model used in my voice project[4]:

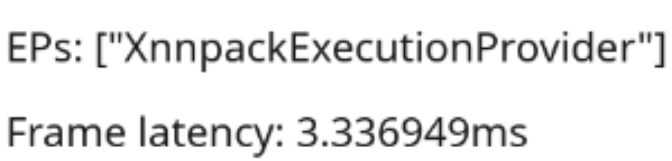


Figure 4: XNNPACK

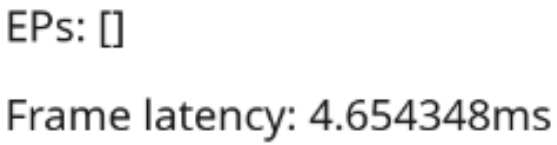


Figure 5: Default CPU

[1] No dGPU

[2] Backends on Strix Halo is a bit unstable

[3] CPU Acceleration, requires compiling ONNX runtime from scratch

[4] <100ms is acceptable for use in the app, so it's kinda useless